GUIA VISUAL 001

O QUE É LINUX

Servidor desconhecido, 3h20 da manhã: `uname -a` *e* `/etc/os-release` *respondem perguntas diferentes.*

001
GUIAS LINUX

LINUX É O KERNEL — O RESTO É INTEGRAÇÃO

ESPAÇO DE USUÁRIO

APLICAÇÕES
navegador · nmap · Python · o seu script

UTILITÁRIOS E SHELL
bash · coreutils · systemd · apt

BIBLIOTECAS
glibc · OpenSSL · libcurl

chamadas de sistema

ESPAÇO DE KERNEL

KERNEL LINUX
processos · memória · arquivos · rede · drivers

HARDWARE
CPU · RAM · disco · placa de rede

O KERNEL DEFINE

Drivers e suporte a hardware, escalonamento de CPU, memória, sockets. **Nenhum programa toca o hardware** — todos pedem ao kernel. Monolítico e **modular**: `lsmod` mostra o que foi carregado sem reiniciar.

A DISTRIBUIÇÃO DEFINE

Biblioteca C, init, shell, **gerenciador de pacotes**, repositório, instalador e a política de atualização. É onde nasce o `apt` × `dnf` — e a janela de suporte.

OS NÚMEROS NÃO CONVERSAM

Kernel `6.12.25` e distro `2025.2` não têm relação. Ler os dois é o **primeiro diagnóstico** de qualquer máquina desconhecida.

AS DUAS PERGUNTAS DO PRIMEIRO MINUTO

O TRONCO DEFINE OS COMANDOS

FAMÍLIA	PACOTE · GERENTE	DERIVADAS	ONDE APARECE
Debian (1993)	`.deb` · `apt`	Ubuntu, **Kali**, Mint, Raspberry Pi OS	Servidor pequeno/médio — a família deste guia
Red Hat (1994)	`.rpm` · `dnf`	Fedora, RHEL, Rocky, Alma	Datacenter corporativo (firewalld, SELinux)
Arch (2002)	`.pkg` · `pacman`	Manjaro	*Rolling* minimalista, montado à mão
Alpine	`.apk` · `apk`	—	Contêiner: usa `musl` no lugar da `glibc`

O QUE SUSTENTA TUDO: AS 4 LIBERDADES

0 Executar para **qualquer** fim

1 Estudar e adaptar (exige o fonte)

2 Redistribuir cópias

3 Distribuir versões modificadas

RESUMO RÁPIDO

- ✓ Linux, no sentido estrito, é **só o kernel**; a distro é o kernel +

NO KALI

Debian rolling da OffSec, saído do ramo `testing`. O `2025.2`

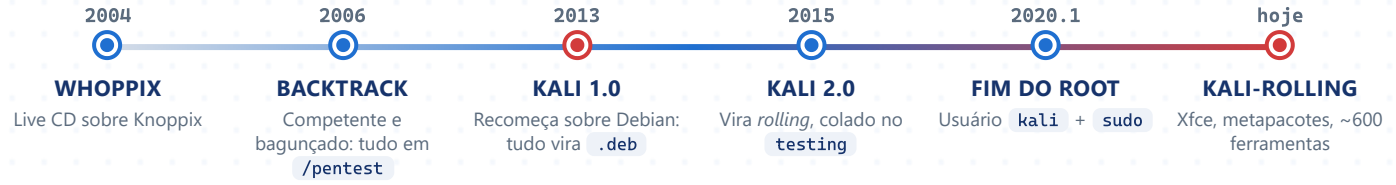
GUIA VISUAL 002

POR QUE KALI LINUX

Instalou Kali no notebook de trabalho e o Wi-Fi morreu no `full-upgrade`? Não é defeito — é uso fora de propósito.

002
GUIAS LINUX

VINTE ANOS ATÉ O `KALI-ROLLING`



AS DECISÕES DE PROJETO QUE EXPLICAM O COMPORTAMENTO

REPOSITÓRIO ÚNICO E ASSINADO

Poucos espelhos, chave GPG do projeto. **Nunca adicione repositório de outra distro** — quebra dependência e destrói a procedência.

CURADAS, NÃO EXAUSTIVAS

~600 ferramentas que passam por triagem (é mantida? licença? duplica?). O oposto do BlackArch, que empacota milhares.

KERNEL COM PATCHES

O sufixo `-kali1` traz **injeção de pacotes e modo monitor** em Wi-Fi. É o que você não obtém instalando as ferramentas à mão.

METAPACOTES

`kali-linux-core` → `default` → `large` → `everything`. Só listas de dependências: VM de 2 GB ou estação de 30 GB.

DISTRIBUIÇÃO DE SEGURANÇA ≠ DISTRIBUIÇÃO SEGURA

Uma instalação padrão traz centenas de binários ofensivos e defaults voltados à conveniência do operador — a superfície de ataque é grande **por construção**. Se comprometem sua máquina Kali, o atacante acha a caixa de ferramentas montada e aquecida. Por isso: VM com snapshot, nada exposto à internet, e um cliente nunca encosta no outro.

O que legitima o uso é a autorização, não a técnica. Sem escopo por escrito e autorização de quem pode dá-la, a mesma ação é crime no Brasil (Lei 12.737/2012).

A MÁQUINA É PARA QUÊ?

USE KALI

- ✓ Teste de invasão **autorizado** e resposta a incidente.
- ✓ Forense digital e engenharia reversa.
- ✓ Auditoria de rede sem fio (é onde o kernel dele brilha).
- ✓ Laboratório: CTF, máquinas vulneráveis, aula prática.
- ✓ Sempre que possível: **em VM com snapshot**.

USE OUTRA

- ✓ Servidor → **Debian estável, Ubuntu LTS, Rocky**. Rolling não tem janela de suporte.
- ✓ Desktop de trabalho → Ubuntu, Fedora, Mint.
- ✓ Só aprender Linux → Debian ou Ubuntu em VM.
- ✓ Se você não sabe o que a ferramenta faz — ou se a empresa proíbe.
- ✓ Vizinhas úteis: Parrot, BlackArch, Tails, REMnux.

RESUMO RÁPIDO

- ✓ Debian *rolling* da OffSec para pentest, forense e auditoria — herdeiro do BackTrack.
- ✓ 2013 reconstruiu sobre Debian para ganhar **empacotamento**

NO KALI

Atualize sempre com `apt update && apt full-upgrade` — o `apt upgrade` se recusa a remover pacotes e deixa a máquina meio atualizada, o pior estado possível num rolling. Se o `apt`

GUIA VISUAL 003

INSTALANDO O KALI

VM, bare metal, live USB, WSL ou contêiner — os cinco caminhos servem a problemas diferentes.

003
GUIAS LINUX

ANTES DE TUDO: A IMAGEM É MESMO DO KALI?

1 — IMPORTE A CHAVE

`wget -qO- archive.kali.org/archive-keyring.gpg | gpg --import`

→

2 — BAIXE AS SOMAS

`SHA256SUMS` e `SHA256SUMS.gpg`, ao lado da imagem.

→

3 — CONFIRA A ASSINATURA

`gpg --verify SHA256SUMS.gpg SHA256SUMS` → tem de dizer **Good signature**.

→

4 — CONFIRA A SOMA

`sha256sum -c SHA256SUMS --ignore-missing` → **OK**. Só então grave.

Conferir só a soma não prova nada — se o espelho foi adulterado, a soma veio adulterada junto. A assinatura GPG é o elo que fecha a cadeia. O aviso `key is not certified` é normal; `BAD signature` não é: descarte e baixe de novo.

OS CINCO CAMINHOS

MÉTODO	ENTREGA	COBRA	USE QUANDO
Máquina virtual ← o padrão	Isolamento, **snapshot**, uma VM por cliente	Wi-Fi só por USB externo repassado; perde desempenho	Estudo e quase todo trabalho profissional. 4 GB RAM · 2 vCPU · **60–80 GB**
Bare metal	Desempenho total, GPU para `hashcat`, Wi-Fi interno	Sem snapshot; particionar apaga o resto	Máquina dedicada à função — nunca o notebook de uso geral
Live USB + persistência	Não escreve no disco da máquina; persistência com LUKS	Lento; gravar a mídia é a operação mais perigosa	Forense, recuperação, trabalho em máquina de terceiros
WSL2 (Windows)	Rápido, integra com o Windows; `kali-win-kex` dá tela	**Kernel é o do WSL**: sem patches, USB irregular, sem modo monitor	Linha de comando, scripts, análise de arquivo. Bom complemento, mau substituto
Contêiner	Sobe em segundos, descartável, ideal em pipeline	Usa o kernel do hospedeiro; péssimo como ambiente interativo	Rodar *uma* ferramenta pontual sem instalar nada

NA VM, O MODO DE REDE É DECISÃO DE SEGURANÇA

NAT · O PADRÃO

VM → host → LAN

Sai para a internet escondida atrás do hospedeiro; **ninguém de fora inicia conexão**. Serve para atualizar e navegar.

BRIDGE · CUIDADO

VM → LAN real

IP próprio no roteador, visível como outra máquina. Necessário quando a VM precisa ser alcançada — e é exatamente o que a política de TI proíbe numa rede corporativa.

HOST-ONLY · O LABORATÓRIO

VM ⇔ alvo ⇔ sem saída

Rede fechada entre as VMs. **Máquina-alvo vulnerável nunca em NAT nem em bridge** — senão seu laboratório vira porta de entrada.

GUIA VISUAL 004

PRIMEIRO CONTATO

Não há "Meu Computador" nem letra de unidade. Não é falta — é outro modelo, e ele começa no firmware.

004
GUIAS LINUX

DO BOOT AO PROMPT: QUEM ENTREGA A TELA

FIRMWARE UEFI / BIOS → **GRUB** escolhe o kernel → **KERNEL** monta a raiz → **SYSTEMD** PID 1, sobe tudo → **GRAPHICAL .TARGET** o alvo do systemd → **LIGHTDM** tela de login → **PAM** autentica → **SESSÃO XFCE** painel, janelas → **TERMINAL** o seu prompt

A camada gráfica é a única dispensável. `systemctl get-default` mostra o alvo; `set-default multi-user.target` entrega um sistema completo sem tela — é o que se faz em servidor. E a saída de emergência quando o Xfce trava é **Ctrl+Alt+F3**: um console virtual ligado direto ao kernel. **Ctrl+Alt+F7** (ou F2) volta — a sessão gráfica continuou rodando o tempo todo.

LENDO O PROMPT CARACTERE A CARACTERE

```
┌──(kali㉿kali)-[~]
└─$
```

Vermelho depois de um comando? Ele falhou — o Zsh do Kali colore o prompt pelo código de saída, poupando um **echo $?** a cada passo.

- **kali** — usuário logado
- **kali** — nome da máquina
- **~** — diretório atual = `/home/kali`
- **$** — comum · **#** = **root**, cada comando pode destruir o sistema

XFCE NÃO É UM PROGRAMA — SÃO PEÇAS QUE SE REINICIAM SOZINHAS

PROCESSO	O QUE DESENHA	SINTOMA DE QUE CAIU
`xfwm4`	Bordas, botões, qual janela fica na frente	Janelas sem barra de título → `xfwm4 --replace &`
`xfce4-panel`	A barra: menu, lista de janelas, relógio, bandeja	Sumiu o painel e o ícone de rede
`xfdesktop`	Papel de parede e ícones da área de trabalho	Fundo preto, sem ícones
`xfsettingsd`	Tema, fontes, cursor, teclado e mouse	Volta ao tema cinza padrão
`xfce4-session`	Coordena tudo e restaura o estado ao relogar	Nada restaura entre logins

OS QUATRO AJUSTES DO PRIMEIRO DIA

RESUMO RÁPIDO

NO KALI

GUIA VISUAL 005

ESTRUTURA DE DIRETÓRIOS (FHS)

Raiz com 96% de uso às 23h: se é `/var/log` a história é uma, se é `/var/cache/apt` é outra.

005
GUIAS LINUX

UMA ÁRVORE SÓ, SEM LETRA DE UNIDADE

`/boot`	Kernel, initrd e o GRUB. Enche depois de algumas atualizações → `apt autoremove`
`/usr`	Programas, bibliotecas e dados estáticos. `/bin`, `/sbin` e `/lib` são só links para cá
`/usr/local`	O que **você** compilou. O `apt` nunca escreve aqui
`/etc`	Configuração, **sempre em texto**, nunca binário. Padrão Debian: fragmentos em `*.d/`
`/home` `/root`	Diretórios pessoais. O root fica fora de `/home` — que pode falhar ao montar
`/opt`	Software de terceiros autocontido, um diretório por produto
`/srv`	Dados servidos a terceiros (no Debian, na prática, `/var/www`)
`/mnt` `/media`	Pontos de montagem para mídia removível
`/var/log`	Logs. O primeiro suspeito no alerta de disco cheio
`/var/lib`	Estado persistente: base do `dpkg`, PostgreSQL, Docker
`/var/cache`	Recriável: `apt clean` libera espaço em 10 segundos
`/tmp`	Curta duração. Costuma ser **tmpfs = RAM**: some no boot
`/var/tmp`	Temporário que **sobrevive** à reinicialização
`/proc`	Processos e kernel. `/proc/self` é quem está lendo
`/sys`	Dispositivos e drivers: temperatura, `operstate` da placa
`/dev` `/run`	`null`, `zero`, `urandom`, discos · e o estado da execução atual

em disco, estático · variável — cresce e enche · virtual — gerado pelo kernel, não ocupa disco

A LÓGICA POR TRÁS · E ONDE INSTALAR O QUE

ESTÁTICO · COMPARTILHÁVEL
`/usr`
Pode ser montado somente-leitura ou servido por rede a várias estações.

VARIÁVEL · COMPARTILHÁVEL
`/srv` · `/var/mail`
Muda, mas faz sentido para outras máquinas.

ESTÁTICO · LOCAL
`/etc` · `/boot`
Específico desta máquina — por isso é o alvo prioritário de backup.

VARIÁVEL · LOCAL
`/var/log` · `/run` · `/tmp`
Volátil e local: exige rotação e monitoramento de espaço.

VEIO DE ONDE	VAI PARA ONDE
Gerenciador de pacotes	`/usr/bin` — **não mexa**
Compilado do fonte	`/usr/local`
Terceiro autocontido	`/opt/<nome>`
Script só seu	`~/.local/bin`
Dados servidos	`/srv` ou `/var/www`

RESUMO RÁPIDO

- ✓ O caminho não diz em que disco o arquivo está — `findmnt --real` e `df -hT` dizem.
- ✓ Disco cheio: `sudo du -xh --max-depth=1 / | sort -rh | head`. O `-x` evita `/proc` e `/sys`.
- ✓ **Nunca escreva à mão em `/usr/bin`**: a próxima atualização sobrescreve sem avisar.
- ✓ `/proc`, `/sys` e `/dev` são "tudo é arquivo" na prática: leia com `cat`.

NO KALI

Acabou o `/pentest/` do BackTrack: **ferramenta é pacote comum em `/usr/bin` — `dpkg -S /usr/bin/nmap` diz de qual pacote veio.** Os dados de cada uma ficam em `/usr/share/<ferramenta>` (scripts NSE, módulos do Metasploit) e as listas em `/usr/share/wordlists` — o `rockyou.txt.gz` vem compactado e precisa de `gunzip` uma vez. Ferramenta clonada de repositório vai para `/opt`. **Atenção ao `/tmp` em RAM:** salvar captura longa ali numa VM pequena derruba a máquina — use `~/lab/capturas`.

GUIA VISUAL 006

ANATOMIA DE UM COMANDO

`rm arquivo com espaco.txt` *não apaga um arquivo: apaga três. Quem separou foi o shell, antes do programa existir.*

006
GUIAS LINUX

A LINHA, PALAVRA POR PALAVRA

`nmap`	`-sV`	`--top-ports`	`100`	`-oN`	`saida.txt`	`192.168.56.0/24`
COMANDO o que será executado	OPÇÃO CURTA um traço, uma letra	OPÇÃO LONGA dois traços, palavra	VALOR pertence à opção anterior	OPÇÃO com valor	VALOR arquivo de saída	OPERANDO o alvo do trabalho

Para o shell, todas as palavras depois do comando são iguais. Quem decide que `-sV` é opção e o resto é alvo é o **programa**, ao ler seu vetor de argumentos — por isso não existe sintaxe universal. Curtas sem valor se agrupam (`ls -lah`); `--` marca o **fim das opções** e salva de nome de arquivo que começa com traço: `rm -- -arquivo-estranho` . E há os rebeldes: `tar xzf` , `ps aux` , `dd if=` … `of=` … — sem traço nenhum.

COMO O SHELL DECIDE O QUE EXECUTAR

1. **PALAVRA-CHAVE** — `if` , `for` , `while` — é sintaxe, não programa
2. **ALIAS** — Apelido seu ou da distro. **NÃO VALE EM SCRIPT**
3. **FUNÇÃO** — Definida na sessão ou no arquivo de init
4. **BUILTIN** — `cd` , `echo` , `export` — dentro do próprio shell
5. **PATH** — Arquivo em disco, na ordem dos diretórios

`type -a <comando>` **mostra qual das cinco venceu** — e explica por que o terminal e o script se comportam diferente. O **diretório atual não está no** `PATH` : por isso `./script.sh` . Não é inconveniência, é segurança — com `.` no `PATH` , um arquivo chamado `ls` num diretório gravável vira escalada de privilégio. **Nunca acrescente** `.` **ao** `PATH` **.**

ZERO É SUCESSO — E É O QUE DÁ SENTIDO AO ENCADEAMENTO

CÓDIGO DE SAÍDA	SIGNIFICA
0	Sucesso — o único valor bom
1	Erro genérico (`grep` sem achar nada)
2	Erro de uso / arquivo inexistente
126	Existe, mas não é executável
127	Comando não encontrado — erro de digitação ou não instalado
130	Interrompido por `Ctrl+C` (128 + sinal 2)

`a && b`
Roda **b** só se **a** deu certo. **O padrão para tudo que pressupõe o passo anterior.**

`a || b`
Roda **b** só se **a** falhou — o tratamento de erro.

`a ; b`
Roda **b** sempre, dê no que der.

`a &`
Roda em segundo plano e devolve o prompt na hora.

`\rm · command rm`
Ignora o alias e chama o programa real.

GUIA VISUAL 020

PERMISSÕES DE ARQUIVO

Quem pode ler, escrever e executar — e por que `chmod 777` não resolve.

020
GUIAS LINUX

ANATOMIA DA SAÍDA DO `LS -L`



TIPO	DONO	GRUPO	OUTROS
arquivo	6	4	4

`r` = ler (4) `w` = escrever (2) `x` = executar (1) → `-rw-r--r--` = `644`

A REGRA QUE QUASE NINGUÉM APRENDE

SÓ UMA CLASSE SE APLICA

O Linux checa na ordem: é o **dono**? senão, está no **grupo**? senão, **outros**. Para na primeira que casa.

EM ARQUIVO

r lê o conteúdo · **w** altera o conteúdo · **x** executa como programa.

EM DIRETÓRIO

r lista nomes · **w** cria/remove itens · **x** *atravessa* o diretório. Sem **x**, nada dentro é acessível.

AS DUAS NOTAÇÕES DO `CHMOD`

SIMBÓLICA

- ✓ Alvo: `u` dono · `g` grupo · `o` outros · `a` todos
- ✓ Operador: `+` adiciona · `-` remove · `=` define
- ✓ Ex.: `chmod u+x script.sh`

OCTAL

- ✓ Soma dos bits por classe: **r=4, w=2, x=1**
- ✓ Três dígitos: dono · grupo · outros
- ✓ Ex.: `chmod 755 script.sh`

NA PRÁTICA

OCTAL	SIMBÓLICO	USO TÍPICO
`644`	`-rw-r--r--`	Arquivo comum (todos leem, só o dono edita)
`600`	`-rw-------`	Arquivo privado — chave SSH, segredo
`755`	`-rwxr-xr-x`	Script/binário e diretórios navegáveis
`700`	`-rwx------`	Diretório pessoal, script só seu (`~/.ssh`)

chmod 777 não é solução de nada. Abre leitura, escrita e execução para *qualquer* usuário da máquina. Se o acesso falha, o culpado quase sempre é a falta de `x` em um diretório do caminho — não a permissão do arquivo.